

# UniVerse: A Trust-Based Campus Community and Skill-Exchange Platform

Project Proposal for UCS503P  
Submitted to: Ms. Paramveer Sidhu  
Thapar Institute of Engineering and Technology

Sehajneet Kaur, CSED\*      Raynee Jindal, CSED<sup>†</sup>      Sahejbir Singh, CSED<sup>‡</sup>  
Shaurya Ranjan, CSED<sup>§</sup>

August 25, 2026

## 1 Higher-order goal

... secondary framing

This project aims to strengthen peer-to-peer trust and knowledge circulation within a college campus by making mentorship, skill-sharing, and resource-lending discoverable to every student, not just those already embedded in a society or a large personal network. While the broader goal is community-building, the immediate engineering objective is to translate *trust-based peer exchange* into a measurable, deployable software solution.

## 2 Time-to-value

... primary heuristic

- We will deliver a usable core loop quickly by prototyping the channel-based community space and the skill-swap matching flow, and validating both with real students within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations and targeting CI-backed automation early to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration – a working set of community channels and a functioning skill-swap points system – then improved based on user feedback.

## 3 Problem Statement

Students at a residential college campus often cannot access mentorship, specific skills, or basic resources (borrowing an item, recovering something lost) unless they already belong to a society or have a wide personal network. Common pain points include:

- **Access inequality:** guidance and mentorship on niche or specific skills is concentrated within existing social circles, leaving unconnected students without a path in.

---

\*Roll No: 1024030215, Email: [skaur10\\_be24thapar.edu](mailto:skaur10_be24thapar.edu)

<sup>†</sup>Roll No: 1024030711, Email: [rjinadal\\_be24thapar.edu](mailto:rjinadal_be24thapar.edu)

<sup>‡</sup>Roll No: 1024030247, Email: [ssingh10\\_be24thapar.edu](mailto:ssingh10_be24thapar.edu)

<sup>§</sup>Roll No: 1024030694, Email: [sranjan\\_be24thapar.edu](mailto:sranjan_be24thapar.edu)

- **Redundant paid learning:** students often pay for external resources to learn something a peer on campus could already teach them.
- **No trust layer for informal exchange:** lending, borrowing, lost-and-found, and general help currently happen through scattered WhatsApp groups with no accountability, history, or verification.
- **No record of informal learning:** peer-taught skills and mentorship sessions leave no trace a student can point to later, even though the effort and outcome are real.

**Why this matters now (contextual relevance):** With a large, skill-diverse student body living on one campus, the supply of mentorship and resources already exists – what is missing is a trusted, discoverable layer to connect it.

## 4 Proposed Solution

*... Software Proposition*

### 4.1 Overview

Build a **web-based** campus community platform, UniVerse, with two core pillars:

- **Community channels:** Discord-style dedicated channels for lending, borrowing, lost items, found items, and general discussion.
- **Skill Swap:** a mentorship and peer-learning marketplace where users list skills they can teach and skills they want to learn, request sessions, and build a points-based reputation.
- **User profile:** name, branch, skills offered/wanted, points balance, sessions taken/given, and leaderboard rank – shareable as proof of learning (e.g., on LinkedIn).
- **Recommendation surface:** the Skill Swap landing page recommends listings based on the skills a user has marked as wanted.

### 4.2 Core workflow

1. Student creates a profile listing skills they can teach and skills they want to learn.
2. Skill Swap surfaces recommended mentors/listings based on the student's "want to learn" list.
3. Student sends a request for a session (guidance, mentorship, or skill swap); once completed, the mentor earns points and the learner spends points.
4. Points scale with uniqueness of mentees taught, feeding a leaderboard that doubles as a portable record of contribution.
5. In parallel, students post and browse lending, borrowing, lost, and found items in dedicated channels, building a visible history of trustworthy exchanges.

### 4.3 Operational constraints for an educational, campus setting

- Keep the solution usable on low-to-mid range devices via responsive web design.
- Avoid dependence on advanced research algorithms (e.g., ML-based matching); focus on workflow, trust mechanics, and system correctness.
- Restrict sign-up to verified college email addresses to keep the trust layer meaningful.
- Design for deployability using common web hosting and managed databases.

## 5 Solution Approach

*... Engineering focus*

This is an engineering course project, so we focus on scalable platform and workflow aspects rather than novel algorithm research.

### 5.1 Points and reputation logic

*... non-research*

The points system will be rule-based, not ML-based:

- Teaching a skill awards more points than learning one costs, by a fixed configurable ratio.
- Teaching additional *unique* learners yields a bonus, discouraging repeated sessions with the same pair from inflating rank.
- All point transactions are logged per session for auditability and dispute resolution; there is no automatic deduction without a completed, confirmed session.

### 5.2 Recommendation logic

Listing recommendations on the Skill Swap page will use a simple tag-matching heuristic (overlap between a user's "want to learn" tags and other users' "can teach" tags), without claiming state-of-the-art personalization.

### 5.3 Web architecture

*... web-based solution*

A typical 3-tier web architecture:

- **Frontend:** React-based responsive UI for channels, profiles, Skill Swap listings, and the leaderboard.
- **Backend API:** Node.js/Express service handling authentication, channel messaging, session requests, and points transactions.
- **Database:** MongoDB storing users, profiles, channel messages, skill listings, sessions, and points history.

### 5.4 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features (skill swap first, then channels, then leaderboard/profile polish).

## 6 Evaluation Criterion

*... measurable and attributable*

We define success using measurable metrics tied to the two core workflows.

## 6.1 Primary evaluation metric

**Session completion rate:** proportion of Skill Swap session requests that result in a confirmed, completed session.

- **Target:**  $\geq 60\%$  of session requests completed during the pilot window.
- **Attribution:** measured from database timestamps and session-confirmation records (request created vs. marked complete by both parties).

## 6.2 Secondary evaluation metrics

- **Unique mentee reach:** average number of distinct learners taught per active mentor.
- **Channel activity:** number of lending/borrowing/lost/found posts resolved (item returned or matched) versus posted.
- **User clarity:** user-reported clarity of the points and ranking system using a simple 1–5 feedback question.
- **Reliability:** availability of staging/production for login, posting, and session requests during peak campus hours (e.g.,  $\geq 99\%$  uptime in pilot).

## 6.3 Pilot validation plan

*... high-level*

- Recruit 20–30 pilot users across multiple branches and years to ensure skill diversity.
- Track requested vs. completed sessions and channel resolutions over the iteration window, comparing against a baseline collected via short interviews on how students currently find mentors or lend/borrow items.

# 7 Scalability

*... with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in number of users, channels, and concurrent Skill Swap requests by:
  - decoupling components (frontend/backend),
  - enabling pagination and indexing for channel messages and skill listings,
  - using stateless API design.
2. **Operationally deployable (practically):** The system should run on common web hosting:
  - managed MongoDB instance,
  - containerized or platform-hosted deployment,
  - CI/CD pipeline for staging/production.

## 8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- React for the frontend; Node.js/Express for REST APIs.
- MongoDB (managed, e.g., Atlas) as the primary document store.
- Token-based authentication restricted to college email domains.
- Object storage for optional attachments (item photos) or local storage for pilot.
- Test framework for unit/integration tests (e.g., Jest).
- CI runner and staging deployment target.

We will use these mature components to focus on workflow design, trust mechanics, and measurement rather than inventing new infrastructure.

## 9 Project scope and deliverables

*... iteration-friendly*

### 9.1 Initial deliverable

*... first iteration*

- User authentication (college email) and basic profile (name, branch, skills)
- Community channels: lending, borrowing, lost, found, general
- Skill Swap: list skills to teach/learn, browse listings
- Session request flow with manual completion confirmation
- Points logic: award/deduct on confirmed session completion
- CI: build + backend unit tests + linting
- CD to staging with a single command or pipeline job

### 9.2 Subsequent deliverables

- Tag-based recommended listings on the Skill Swap landing page
- Leaderboard with rank, sessions taught/taken, and unique-mentee bonus
- Shareable profile summary (exportable for LinkedIn)
- Improved search/filtering and indexed queries for scale readiness

## 10 Risks and mitigations

- **Trust and safety:** restrict sign-up to verified college emails; keep session and item-exchange history visible on profiles to discourage bad behavior.
- **Points gaming:** require both-party confirmation before awarding/deducting points; cap bonus effects; log all transactions for audit.
- **Low initial supply of mentors:** seed the platform with a founding cohort of students across branches before wider launch.

- **Evaluation measurement ambiguity:** instrument timestamps and define clear session/channel-resolution states before development begins.
- **Operational issues in pilot:** use staging early; ensure CI/CD produces repeatable deployments.

## 11 Summary

This project proposes UniVerse, a deployable campus web platform that reduces the friction of finding mentorship, learning niche skills, and trusting peer-to-peer lending and lost-and-found exchange. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially session completion rate). It remains focused on engineering workflow, trust mechanics, and scalability readiness rather than research-driven novelty.